

Islamic University of Technology

Lab: Project Selection Engineering

CSE 4810: Algorithm Engineering

Group 4 — Project Selection Problem (Open-Pit Mining)

ASH, ABT(PT)

August 6, 2026

General Instruction

This lab consists of **one focused task** and is designed to fit a **1.5-hour** session. Written derivations are replaced by **live implementation, profiling, and verbal defense**. You must:

- Step 1.** Complete the `// TODO` sections in the provided C++ source file — this includes both **building the flow network** and **implementing Max-Flow from scratch**.
- Step 2.** Compile and run the program, feeding it the exact sanity-check and benchmark inputs specified.
- Step 3.** Produce the requested live artifact — an optimal net profit, a selected-block set, and a runtime plot — and keep your terminal/IDE open.
- Step 4.** Be ready to defend your engineering choices verbally using the artifact on screen; no written report is collected.

Modeling + Max-Flow Implementation ~55 minutes

Sanity Check & Verification ~15 minutes

Live Profiling ~20 minutes

This is the heaviest single-task lab you will encounter — it combines problem modeling AND a from-scratch Max-Flow implementation. Read the entire task before writing any code; a wrong network model makes a correct Max-Flow implementation useless.

Files Provided

<code>ProjectSelection.cpp</code>	Network-construction + Max-Flow skeleton
<code>plotter_group4.py</code>	Plots runtime data

Task: Project Selection Problem (Open-Pit Mining) via Max-Flow Min-Cut

Objective: Given a set of blocks with profits/costs and precedence constraints, model the problem as a flow network and solve it with a from-scratch Max-Flow implementation to find the maximum-profit feasible selection.

The Scenario

MetroCore Mining Co. has surveyed a small open-pit site and divided the ground into discrete **extraction blocks** based on a geological survey. Each block is a chunk of ground that can be dug out independently, but with one physical restriction: **you cannot extract a block until every block that sits above it or beside it (blocking access) has already been extracted.** You cannot tunnel through solid rock to reach a pocket of ore — you have to clear a path to it first, top-down and outside-in, exactly like real open-pit excavation. This is why each block comes with a list of **prerequisite blocks** that must be removed first.

Every block also has a **net value** once it is accessible:

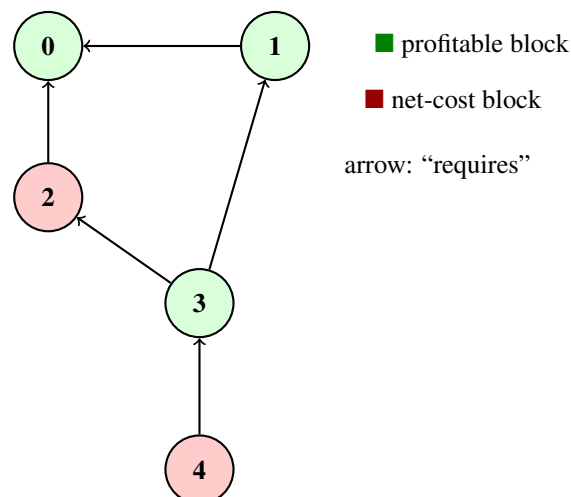
- Some blocks contain enough ore that the revenue from selling it exceeds the cost of digging it out — these are **profitable blocks** (positive value).
- Other blocks are pure overburden, low-grade waste rock, or require expensive extra work (shoring, dewatering, blasting through a fault line) — these are **net-cost blocks** (negative value), but sometimes you still have to remove them anyway, because a profitable block sits behind them.

MetroCore’s question is simple to state and hard to brute-force: **which subset of blocks should be extracted to maximize total net profit, subject to every selected block’s prerequisites also being selected?** Selecting every profitable block blindly can be wrong (you might drag in a costly block you didn’t need to), and selecting nothing is always safe but leaves money on the table. With even a few dozen blocks, checking every valid subset by hand is hopeless — this is exactly the kind of combinatorial selection problem Max-Flow Min-Cut solves efficiently.

The Five-Block Site (used in your sanity check)

Block	Description	Net Value	Requires (must be removed first)
0	Surface ore cap — immediately accessible	+10	(none)
1	North vein, behind the surface cap	+8	Block 0
2	Waste shoulder beside the cap	−3	Block 0
3	Deep ore pocket, boxed in by both sides	+6	Blocks 1 and 2
4	Fractured zone beyond the deep pocket	−20	Block 3

Block 3’s ore pocket is only exposed once *both* the north vein (1) and the waste shoulder (2) are cleared — it is physically boxed in on both sides. Block 4 lies beyond block 3, but hitting it means crossing a fractured zone that requires expensive shoring; its cost (20) outweighs anything it could unlock, so a profit-maximizing mine plan digs out {0, 1, 2, 3} and leaves block 4 in the ground.



From Story to Numbers

Your C++ code never sees "ore" or "waste rock" — it only sees two arrays, and your first real modeling job is translating the story above into exactly this:

```
1 vector<long long> value      = {10, 8, -3, 6, -20};
2 vector<vector<int>> precedence(5);
3 precedence[1] = {0};          // Block 1 requires Block 0
4 precedence[2] = {0};          // Block 2 requires Block 0
5 precedence[3] = {1, 2};       // Block 3 requires Blocks 1 AND 2
6 precedence[4] = {3};          // Block 4 requires Block 3
```

This exact array pair is already wired into `sanity_check()` in the skeleton file — you do not need to type it yourself. Your job is to make `build_matching_network` (Group 3) style thinking apply here too: turn *this array pair* into a flow network, the same way you would turn any other input into a graph.

Key Idea — Project Selection as Min-Cut

Every block i has a net value: positive means profit if extracted, negative means cost if extracted. Build a network with edge **source** $\rightarrow i$ of capacity $value[i]$ for profitable blocks, and edge $i \rightarrow$ **sink** of capacity $-value[i]$ for costly blocks. For every precedence constraint " i requires j ", add an **infinite-capacity** edge $i \rightarrow j$. A min-cut can never sever an infinite edge, so if i ends up on the source side of the cut (selected), j is forced to be on the source side too (also selected) — precedence is automatically enforced. The optimal net profit is: (total available profit) $-$ (min-cut value).

0.1 TODO 1: Implement `bfs_augmenting_path(source, sink, parentEdge)`

[Hint] Standard BFS, traversable only along edges with $capacity - flow > 0$. Record `parentEdge[v] = {u, edge_index}` for every visited vertex.

0.2 TODO 2: Implement `max_flow(source, sink)`

[Hint] Loop: call `bfs_augmenting_path`. If it fails, stop. Otherwise find the bottleneck along the path (minimum residual capacity), push that much flow onto every edge on the path, and subtract it from every reverse edge. Accumulate the bottleneck into the total.

0.3 TODO 3: Implement `find_selected_set(source, numBlocks)`

[Hint] Call this only after `max_flow` has fully terminated. BFS/DFS from `source` using only edges with positive residual capacity to find the reachable set S . The selected blocks are exactly the vertices in S other than the source itself — remember vertex $i + 1$ corresponds to block i .

0.4 TODO 4: Implement `build_project_selection_network(value, precedence)`

[Hint] Vertex 0 is the source, vertex `numBlocks + 1` is the sink, and block i is vertex $i + 1$. Add `source  $\rightarrow$  vertex( $i$ )` with capacity $value[i]$ when $value[i] > 0$; add `vertex( $i$ )  $\rightarrow$  sink` with capacity $-value[i]$ when $value[i] < 0$; add `vertex( $i$ )  $\rightarrow$  vertex( $j$ )` with capacity `INF_CAP` for every j in `precedence[i]`.

Sanity Check — Expected Output

Run your code on the five-block MetroCore site from **The Scenario** above (already wired into `sanity_check()` — no need to re-enter it). You should see:

```

1 --- PROJECT SELECTION SANITY CHECK (5 blocks) ---
2 Total available profit : 24
3 Min-cut value         : 3
4 Optimal net profit     : 21 (Expected 21)
5 Selected blocks       : 0 1 2 3 (Expected 0 1 2 3)
6 Precedence constraints satisfied: YES
7 -----

```

Compile & Run

```

1 g++ -O2 -std=c++17 -o mining ProjectSelection.cpp
2 ./mining

```

Live Profiling

Runs to Perform

- `run_benchmark()` (already wired into `main`) generates a synthetic open-pit "mining grid" (shallow rows = overburden cost, deep rows = ore profit, each block depending on the three blocks directly above it) at increasing grid sizes, and times your Max-Flow solve. Run it and inspect `data_mining.txt`.
- **Plot it:** `python3 plotter_group4.py data_mining.txt` produces `mining_runtime.png`.

Core Engineering Defense (Verbal Check)

- **Why Infinite Capacity?** Explain what would go wrong (in terms of the min-cut construction) if a precedence edge $i \rightarrow j$ were given a large but *finite* capacity instead of `INF_CAP`.
- **Reading the Cut:** Using your sanity-check output, explain concretely why the min-cut value (3) corresponds exactly to "the cost of the cheapest cost-block that must be accepted to unlock enough profit blocks."
- **Grid Structure:** Point to your runtime plot and discuss how the "3 blocks above" precedence pattern in the mining grid affects the number of augmenting paths Edmonds-Karp needs compared to a grid with no precedence constraints at all (i.e., pure knapsack-like selection).

The instructor may ask you to change Block 4's value from -20 to -4 in the sanity-check example and predict, *before* re-running your code, whether Block 4 (and therefore Block 3's full chain) becomes worth selecting. Justify your answer by comparing the revised cost against the profit it would unlock.